Personalized Networking Assistant

Project Documentation

Date	14 July 2026
Team ID	
Project Name	Personalized Networking Assistant
Document Type	Final Project Documentation

Project Name: Personalized Networking Assistant

Overview: An AI-powered full-stack web application that helps users prepare for professional networking through personalized conversation starters, pitches, event-specific dialogue strategies, factual verification, history, feedback, and analytics.

Actual Stack: React, TypeScript, Vite, Node.js, Express, SQLite3, Google Gemini, Wikipedia REST API, Tailwind CSS, Motion, Recharts, bcryptjs, esbuild and Render.

Repository: thishitha06-sketch / Personalized-Networking-Assistant

Technical Architecture:

1. Introduction

Professional networking is difficult when users do not know how to begin conversations, identify relevant topics, or verify information before speaking with new contacts. The Personalized Networking Assistant combines professional profile context, event inputs, factual lookup, and generative AI in one web application.

Users can create secure profiles, enter event or topic context, verify factual information through Wikipedia, generate tailored networking scripts, save previous generations, and submit feedback. Generated content is organized around practical situations such as lobbies, elevators, coffee breaks, speaker transitions, pitches, and follow-up communication.

SMARTBRIDGE
Let's Bridge the Gap

SKILL WALLET
A SmartBridge Product

Project Workflow:

2. Problem Statement

Professionals, students, and conference attendees often experience uncertainty while initiating conversations. Generic advice is usually disconnected from a user's profession, role, career goals, interests, target company, event topic, or the person being approached.

The project addresses this gap by providing personalized and context-aware networking assistance. It also reduces the risk of discussing inaccurate information through an on-demand Wikipedia factual verification step before AI generation.

- Store user feedback.
- Maintain networking recommendations.

3. Objectives

• Provide secure registration, login, and professional profile customization.
• Use profession, role, company, career goals, and interests for personalization.
• Support configurable Basic, Executive, and Enterprise-oriented plan selection.
• Verify relevant topics using Wikipedia.
• Generate personalized networking content with Google Gemini.
• Organize output across 22 practical networking categories.
• Store generation history and event metadata in SQLite.
• Collect likes, dislikes, star ratings, and comments.
• Present generation and satisfaction metrics through an analytics dashboard.
• Keep API credentials on the server and use parameterized database operations.

4. Existing System and Limitations

Traditional networking preparation depends on generic articles, static templates, manual web searches, or general-purpose chatbots. These approaches often require users to switch between tools and repeatedly explain their context.

Common limitations include weak personalization, no integrated factual verification, no structured persistence of previous networking sessions, and no feedback-driven analytics. The proposed system combines these capabilities in one workflow.

5. Proposed Solution

The solution is a full-stack TypeScript application. The React frontend provides onboarding, profile management, event input, factual lookup, AI-generated networking content, history, feedback, and analytics.

The Node.js and Express backend contains server-side routes and services. Gemini performs personalized content generation, Wikipedia provides factual abstracts, and SQLite stores users, sessions, history, and feedback. Sensitive API credentials remain on the server.

6. Functional Requirements

FR1: Secure user registration, login, and professional profile customization.

FR2: Multiple onboarding and plan-selection tiers with configurable premium inputs.

FR3: Wikipedia factual verification for event-specific topics and backgrounds.

FR4: Gemini-powered personalized networking generation across 22 categories.

FR5: Persistent connection and generation history in SQLite.

FR6: Likes, dislikes, ratings, comments, and analytical dashboard metrics.

7. Non-Functional Requirements

Security: Server-side environment variables protect the Gemini API key; passwords use bcryptjs; SQL operations are parameterized.

Usability: Clear onboarding, loading states, readable generated categories, copy actions, history, and feedback controls.

Maintainability: Reusable React components, shared TypeScript types, Express routing, and independent backend services.

Deployability: npm-based development and production workflows with Render deployment and optional persistent SQLite storage.

8. Technology Stack

Frontend: React 18+, TypeScript, Vite, Tailwind CSS, Lucide Icons, Motion and Recharts.

Backend: Node.js, Express, TypeScript, tsx and esbuild.

Database: SQLite3.

AI: Google Gemini through the @google/genai TypeScript SDK.

Factual Integration: Wikipedia REST API.

Security: bcryptjs and environment-variable secret isolation.

Deployment: Render with Node runtime and optional persistent disk storage.

9. System Architecture

The application follows a three-layer architecture.

Presentation Layer: React and TypeScript browser client for profiles, event inputs, factual lookup, generated content, history, feedback, and analytics.

Application Layer: Node.js and Express server with user, Gemini, fact-check, routing, and database services.

Data and External Services: SQLite stores application records; Gemini provides generative AI; Wikipedia provides factual background information.

The browser never receives the Gemini API key.

10. Data Flow

1. The user registers or logs in and maintains a professional profile.
2. Account and profile data are stored in SQLite.
3. The user enters event, company, topic, or networking context.
4. The server can query Wikipedia and return a factual snippet.
5. The backend combines profile context, event input, and relevant factual context for Gemini generation.
6. Gemini returns structured suggestions across practical networking categories.
7. The application displays and stores the generated result.
8. Ratings, likes, dislikes, and comments are persisted as feedback.
9. Stored records contribute to dashboard metrics.

11. Database Design

The SQLite database **networking_assistant.db** stores four core relational areas.

users: User IDs, emails, hashed passwords, professional fields, goals, interests, and plan-related configuration.

sessions: Active local authentication session information.

history: Completed generation details, event inputs, metadata, and generated categories.

feedback: Star ratings, likes, dislikes, comments, and feedback linked to generation history.

The database service initializes required tables and uses parameterized operations.

12. AI and Factual Verification

The application uses two complementary intelligence sources.

Wikipedia Verification: An on-demand server-side query retrieves factual abstracts related to event niches, technical topics, companies, or discussion subjects.

Gemini Generation: The server constructs a personalized prompt using professional profile data, interests, goals, event context, and relevant factual context. Gemini produces structured networking content across 22 categories.

This separates factual lookup from creative language generation while keeping both in one workflow.

13. Major Implemented Features

1. **User Profile and Registration Engine** – Secure accounts and professional profile customization.

2. **Interactive Plan Selection** – Basic, Executive, and Enterprise-oriented onboarding choices.

3. **Real-Time Wikipedia Fact-Checker** – Live factual lookup for event or topic preparation.

4. **22-Category AI Generative Script Engine** – Personalized starters, pitches, event scenarios, and follow-up content.

5. **Connection History Logging** – Persistent generation records and event metadata.

6. **Feedback and SaaS Analytics** – Ratings, likes, dislikes, comments, and dashboard metrics.

14. Implementation Structure

The repository is organized as a modular full-stack application.

src/ contains the React frontend, reusable components, and shared TypeScript structures.
server/ contains backend services and integrations.
api/ supports API-related deployment structure.
server.ts is the main Express server entry point.
package.json defines npm dependencies and scripts.
vite.config.ts configures the frontend build.
.env.example documents required environment variables without exposing secrets.

Academic project deliverables are stored in dedicated phase folders.

15. Security Considerations

The browser does not receive the **GEMINI_API_KEY**; credentials are loaded from server-side environment variables. Passwords are protected using bcryptjs hashing before database storage, and SQL operations use parameterized inputs.

The repository includes an environment template rather than real credentials. Actual secrets should never be committed to GitHub. Production secrets are configured through the hosting platform's environment settings.

16. Testing Approach

Testing covers registration and login, profile creation, plan selection, Wikipedia lookup, Gemini generation, rendering of generated categories, copy interactions, SQLite persistence, history search and deletion, feedback persistence, production builds, and deployment behavior.

External-service response times depend on network conditions and provider availability. The modular service architecture allows AI, factual lookup, and database behavior to be checked independently as well as through the complete user workflow.

17. Results and Discussion

The completed solution integrates the major planned modules into one full-stack application. Users can create professional profiles, prepare event context, retrieve factual background information, generate personalized networking content, revisit previous generations, and provide feedback.

The 22-category structure improves practical usability by organizing outputs around real networking situations instead of returning one generic paragraph. SQLite persistence provides continuity, while the feedback and analytics workflow supports evaluation of user satisfaction.

Activity 5.3: Cloud Deployment Using Render

18. Deployment and Execution

Prerequisites: Node.js v18 or v20 LTS and npm.

Install: npm install

Environment: Configure GEMINI_API_KEY, DATABASE_URL, and NODE_ENV in a local .env file.

Development: npm run dev

Production build: npm run build

Production start: npm run start

For Render deployment, use a Node web service. Persistent SQLite storage can be mounted at **/var/data** with **DATABASE_URL=/var/data/networking_assistant.db**.

Exploring the Website's Web Pages:

Home / Generator Page:

19. Limitations and Future Scope

Current operation depends on internet access for Gemini generation and Wikipedia factual lookup. SQLite is suitable for the present project scale, while larger multi-user deployments may benefit from a managed relational database such as PostgreSQL.

Future improvements can include richer event integrations, calendar synchronization, additional trusted factual sources, voice or text-to-speech interaction, expanded analytics, advanced recommendation logic, and enterprise-scale authentication and database infrastructure.

Results Section:

The AI Recommendation Page displays AI-generated conversation starters, recommended networking

20. Conclusion and References

The Personalized Networking Assistant demonstrates how generative AI, factual verification, professional profile context, and persistent application data can support meaningful networking preparation. The project combines secure profiles, plan selection, Wikipedia verification, a 22-category Gemini generation engine, history, feedback, and analytics in a modular full-stack architecture.

References:

• Personalized Networking Assistant repository and project documentation.
• React and TypeScript documentation.
• Node.js and Express documentation.
• SQLite documentation.
• Google Gemini / Google Gen AI SDK documentation.
• Wikipedia REST API documentation.
• Vite and Render deployment documentation.